Project's Report on

CI/CD Pipeline & Kubernetes Deployment for Marketing Planner Agent

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Akhilesh Tiwari	EN22CS301085
2	Alfez Khan	EN22CS301102
3	Aditi Gaikwad	EN22CS301040
4	Akshat Sakalye	EN22CS301093
5	Aanchal Chaturvedi	EN22CS301014
6	Aditya Patidar	EN22CS301060

Group Name: Group 06D1

Project Number: DO-06

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

1 Problem Statement & Objectives

1.1 Problem Statement

In modern software engineering, deploying multi-tier AI applications involving separate frontend, backend, and database components manually leads to environment inconsistencies, complex Cross-Origin Resource Sharing (CORS) issues, and high margins for human error. Furthermore, traditional deployment methods often require hardcoding sensitive API keys on production servers, creating significant security vulnerabilities. The absence of an automated CI/CD pipeline and container orchestration results in delayed code integration and downtime during updates. This project aims to solve these issues by designing a robust, GitOps-driven CI/CD pipeline for a Marketing Planner Agent. By leveraging Docker, GitHub Actions, and Kubernetes, the project ensures seamless automation, zero-downtime deployments, and the secure, memory-based injection of sensitive LLM API keys.

1.2 Project Objectives

The primary objectives of this project are:

- **To automate code integration using Git:** Implementing a trunk-based development approach where updates to the main branch are version-controlled and integrated efficiently.
- **To implement Continuous Integration (CI):** Automating the build process via GitHub Actions to compile the React frontend and Python backend independently.
- **To containerize the application using Docker:** Packaging the Vite/React frontend (using an Nginx reverse proxy) and the FastAPI/CrewAI backend into portable Docker images to guarantee environment consistency.
- **To implement Continuous Deployment (CD):** Pushing built images to Docker Hub and triggering automatic deployments via secure SSH to an AWS EC2 instance.

- **To deploy the application using Kubernetes:** Managing container orchestration using Minikube on EC2, ensuring high availability, and resolving CORS issues via internal cluster networking.
- **To implement GitOps Secret Management:** Dynamically injecting OpenRouter, Groq, and MySQL database credentials into the Kubernetes cluster memory via ConfigMaps, ensuring no physical .env files reside on the EC2 host.

1.3 Scope of the Project

The scope of this project includes:

- Development of a CI/CD pipeline triggered by GitHub pushes.
- Docker-based containerization of a multi-tier application (React UI, FastAPI Backend).
- Deployment onto a Minikube Kubernetes cluster hosted on AWS EC2.
- Configuration of an Nginx reverse proxy within the frontend container to route /api traffic to the backend.
- Dynamic handling of database URLs and API keys using imperative kubectl commands over SSH.

1.4 System Limitations:

- Limited to a single-node Kubernetes cluster (Minikube).
- No advanced cluster monitoring (e.g., Prometheus/Grafana) implemented initially.

2 Proposed Solution

2.1 Key Features

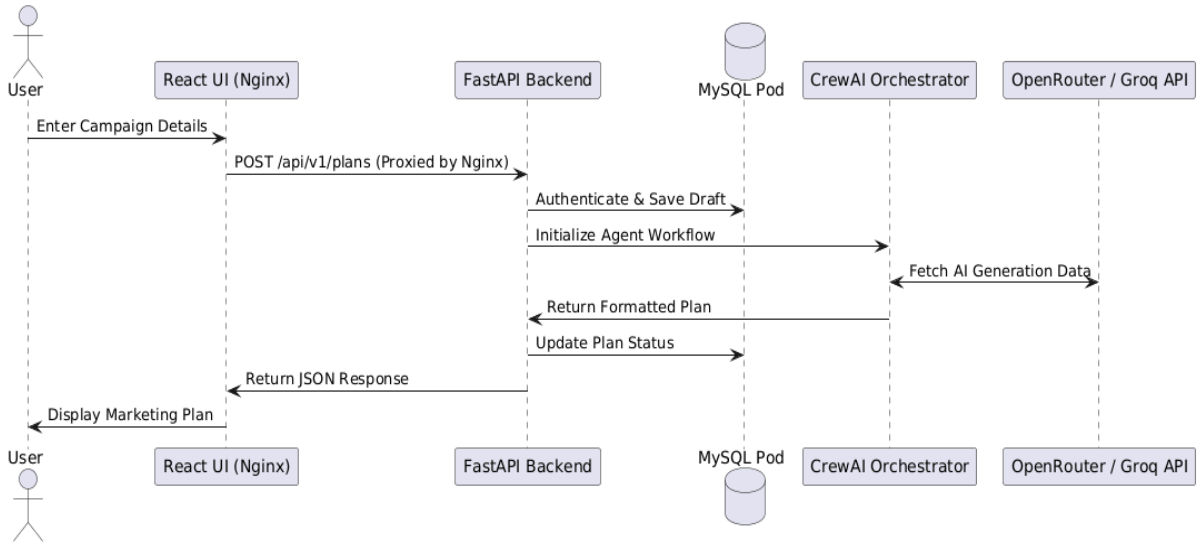
- **Automated CI/CD Pipeline:** Triggered automatically on main branch pushes using GitHub Actions.
- **Zero-Downtime Deployments:** Achieved through Kubernetes rolling updates and imagePullPolicy: Always.
- **Backend-for-Frontend Proxy Pattern:** Nginx elegantly routes API calls, completely eliminating CORS preflight errors in the browser.

- **Secure Secrets Management:** API keys are stored in encrypted GitHub Secrets and injected dynamically into Kubernetes RAM, never touching the host hard drive.

2.2 Overall Architecture/Workflow

2.2.1 Application Workflow

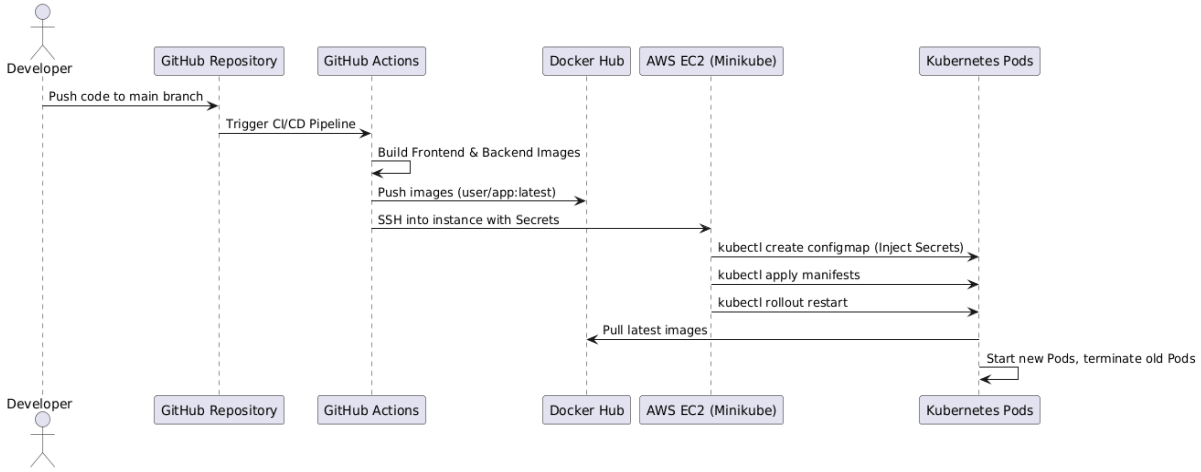
1. **User Input:** The user accesses the React UI and inputs their marketing campaign parameters.
2. **API Routing:** The request hits the Nginx server on port 8080, which intercepts the /api route and forwards it internally to the FastAPI backend on port 8000.
3. **Data Persistence:** FastAPI securely authenticates the user and logs the request in the MySQL database pod.
4. **LLM Processing:** The CrewAI orchestrator triggers the agents, communicating with external LLMs via the OpenRouter/Groq APIs to generate the marketing plan.
5. **Delivery:** The completed marketing plan is returned through the Nginx proxy to the React frontend for display.



2.2.2 DevOps Workflow

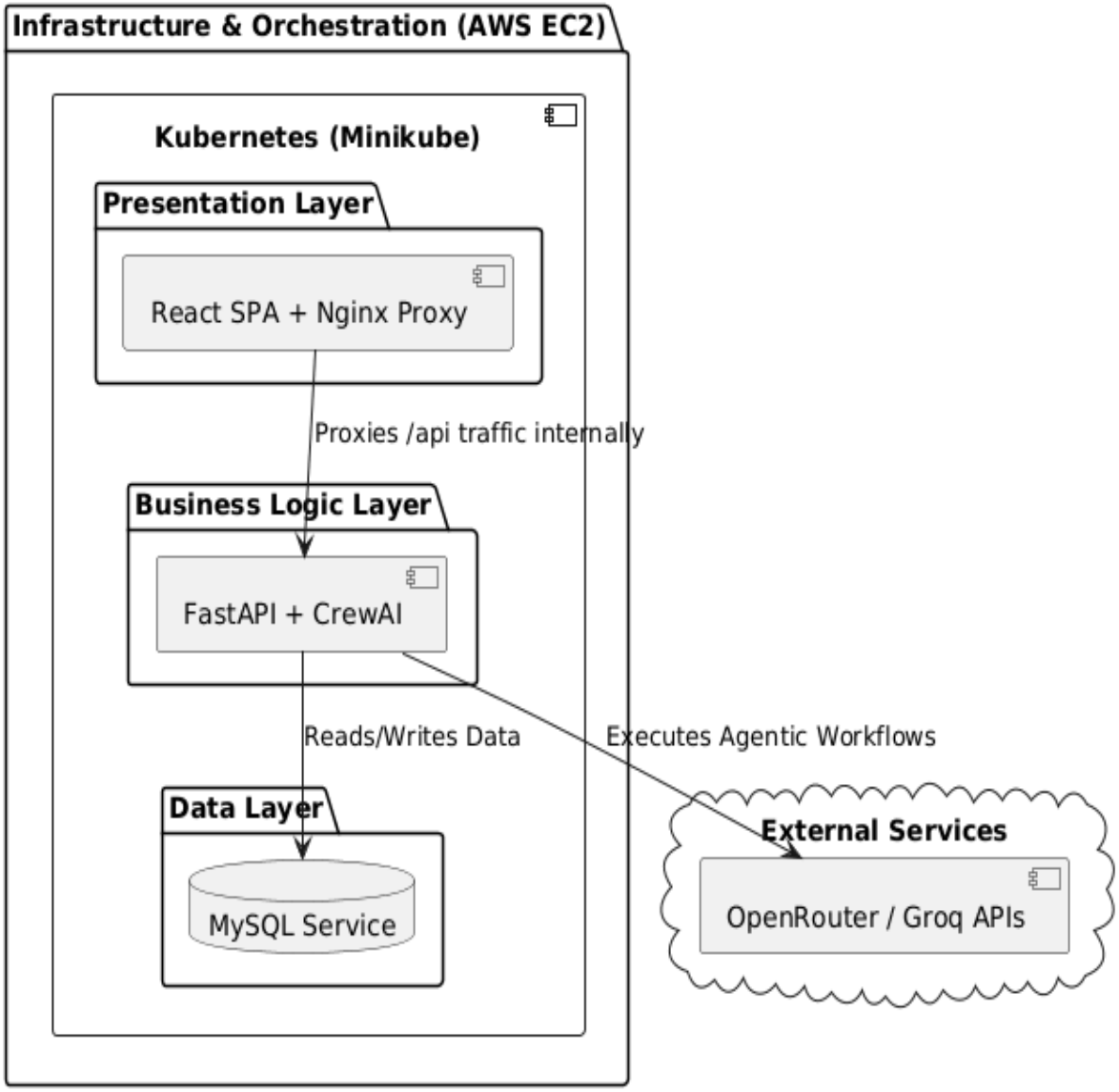
1. **Code Commit:** Developer pushes code changes (e.g., updating the Vite API URL) to the GitHub main branch.

2. **CI Pipeline Initiation:** GitHub Actions runner initializes and builds both the frontend and backend Docker images.
3. **Docker Push:** Images are tagged as :latest and pushed to the Docker Hub registry.
4. **CD Pipeline Initiation:** GitHub Actions establishes a secure SSH connection to the AWS EC2 instance, passing encrypted secrets as temporary variables.
5. **Kubernetes Deployment:** An imperative kubectl create configmap command executes, writing the secrets directly to cluster memory. A kubectl rollout restart is triggered to pull the fresh images.



2.2.3 Architecture Explanation

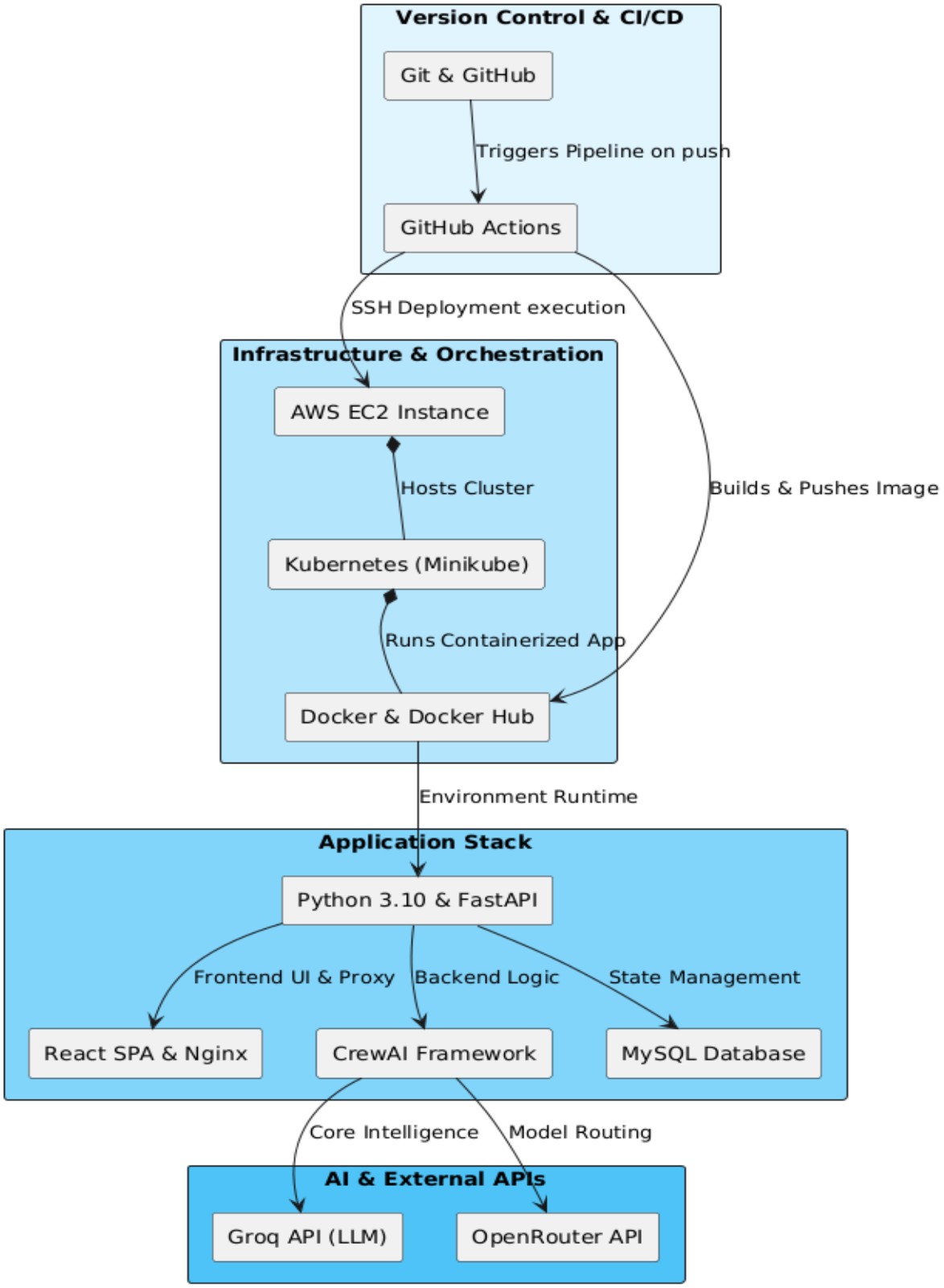
The system follows a decoupled, container-based architecture. The presentation layer (React) is compiled into static assets and served by Nginx, which doubles as a reverse proxy to route traffic to the Python backend. The backend manages state via a dedicated MySQL clusterIP service. Docker ensures the environment mapping is identical across development, CI, and production. Kubernetes manages the lifecycle of all three tiers, ensuring self-healing and managing external access via port-forwarding/NodePort.



2.3 Tools & Technologies Used

- **Git & GitHub:** Version control and trunk-based repository management.
- **Docker & Docker Hub:** Multi-stage containerization and image registry.
- **GitHub Actions:** CI/CD pipeline automation and secret injection.
- **Kubernetes (Minikube):** Container orchestration, ConfigMap memory management, and self-healing.
- **AWS EC2:** Cloud hosting infrastructure and Security Group management.
- **React & Vite:** Frontend Single Page Application framework.
- **Nginx:** High-performance static file serving and reverse proxying.
- **Python (FastAPI):** High-performance backend routing and API management.

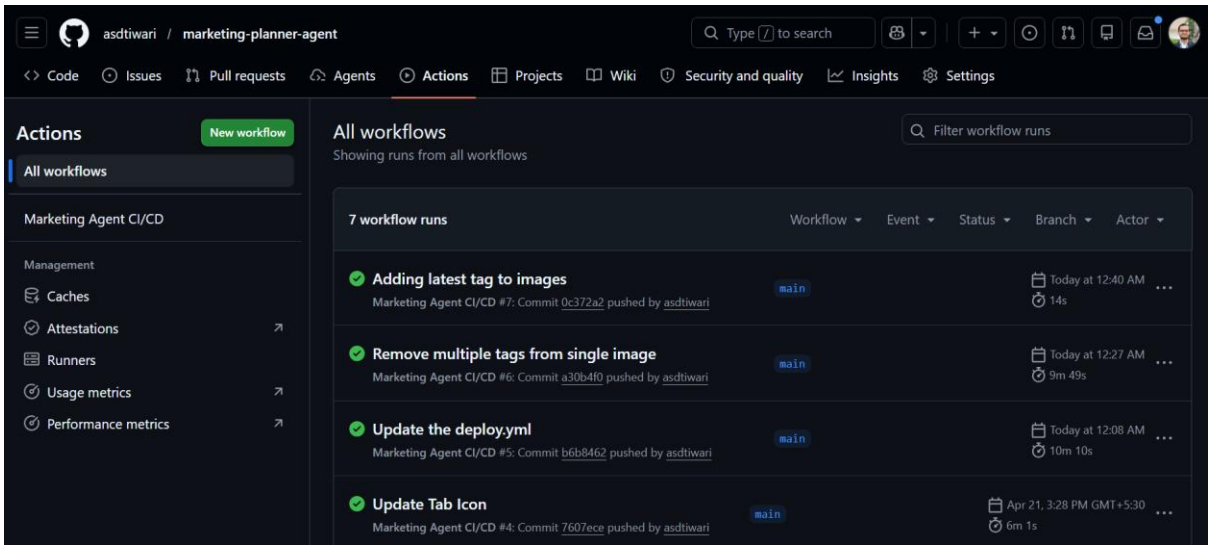
- **CrewAI:** Multi-agent orchestration framework.
- **MySQL:** Relational database for persistent user and plan history.



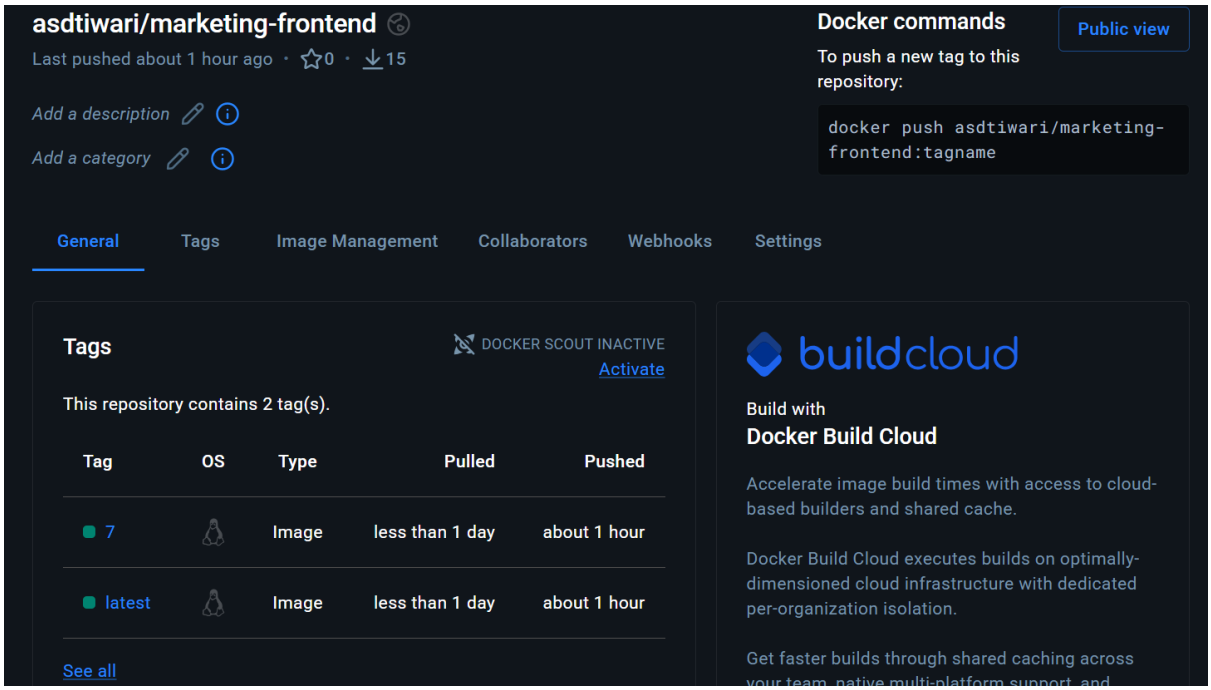
3 Results & Output

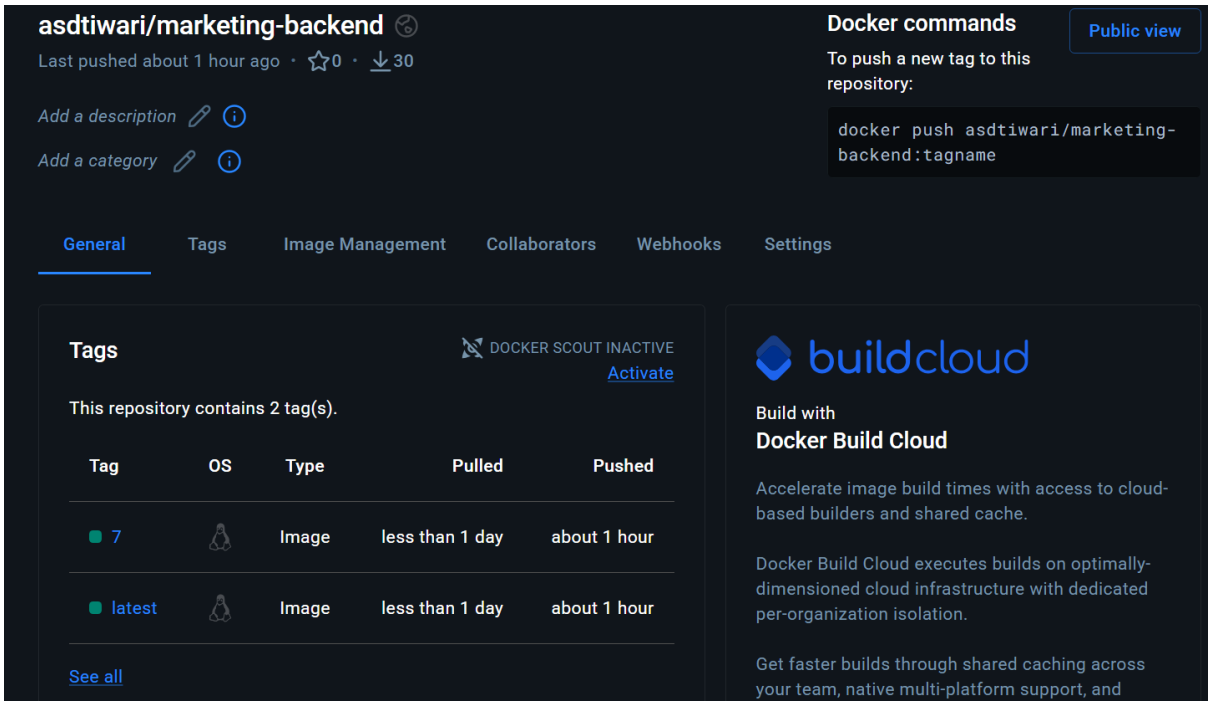
3.1 Screenshots / outputs

3.1.1 Figure 1: GitHub Actions CI/CD Pipeline Success

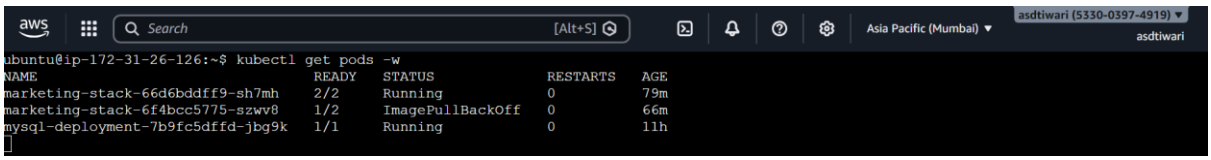


3.1.2 Figure 2: Docker Hub Image Registry

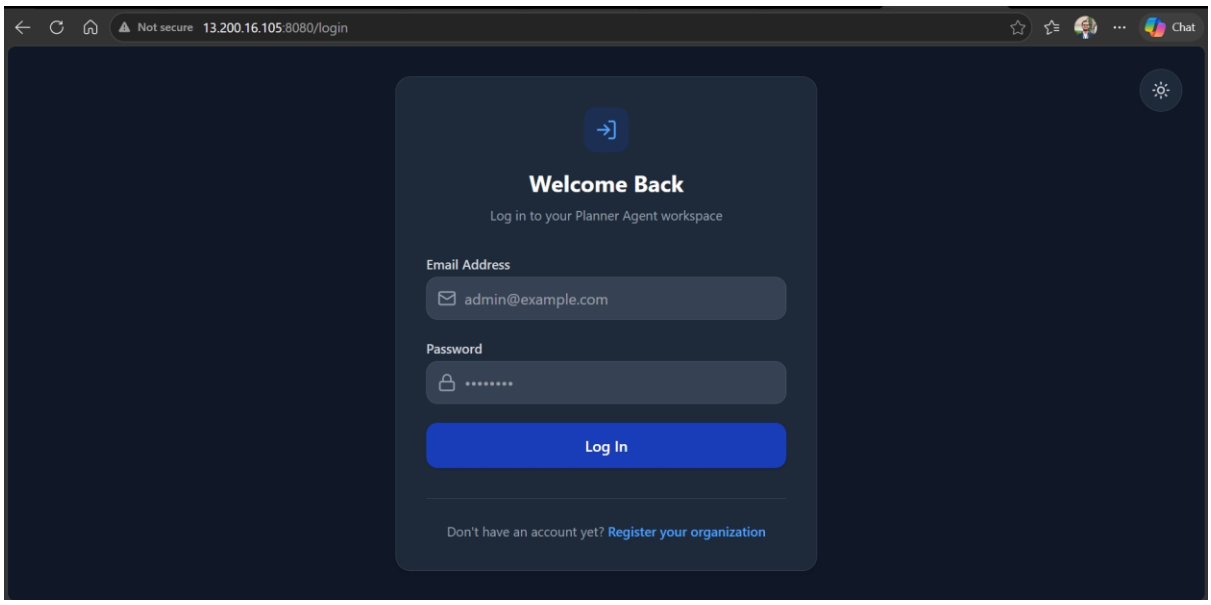




3.1.3 Figure 3: Kubernetes Pods Running on EC2



3.1.4 Figure 4: Application Running (React UI)



3.2 Key outcomes

- **Successfully Implemented CI/CD:** Code changes pushed to GitHub are now automatically built into Docker images and deployed to the AWS server without any manual file transfers.
- **Achieved Zero-Downtime Deployments:** Kubernetes rolling updates allow the backend and frontend to update their code independently without disrupting active user sessions.
- **Secured Credentials:** Sensitive OpenRouter, Groq, and Database connection strings are effectively isolated from the codebase. The GitOps pipeline injects them directly into RAM, preventing unauthorized access.
- **Eliminated Network Errors:** The integration of the Nginx reverse proxy successfully resolved all Cross-Origin (CORS) restrictions, allowing seamless communication between the React SPA and the FastAPI backend.

4 Conclusion

The implementation of the CI/CD pipeline and Kubernetes deployment for the Marketing Planner Agent successfully demonstrates a complete, modern DevOps lifecycle. By integrating GitHub Actions, Docker, and Kubernetes, the project achieved efficient automation, reliable scaling, and highly secure configuration management. The system highlights the viability of deploying decoupled, multi-tier AI applications utilizing modern reverse-proxy patterns, significantly reducing manual deployment errors and creating a highly portable, resilient application environment.

5 Future Scope & Enhancements

- **Automated Testing Gate:** Integration of testing jobs (e.g., PyTest for the backend, Jest for the frontend) into the GitHub Actions workflow to block broken code from being containerized.
- **Cloud-Native Migration:** Transitioning from Minikube (optimized for local development) on EC2 to a fully managed Kubernetes service like AWS EKS (Elastic Kubernetes Service) for production-grade load balancing.

- **Cluster Monitoring:** Integration of Prometheus and Grafana for real-time tracking of pod CPU/RAM usage (critical for AI workloads) and API latency.
- **Helm Charts:** Packaging the Kubernetes YAML manifests into Helm charts to manage versions, dependencies, and rollbacks more efficiently as the application scales.